DECIDABLE LANGUAGE ANALYZER

A PROJECT REPORT

Submitted by

REGNO	NAME
111624102215	SURENDAR HARIPRASAD
111624102219	E THAMILNIDHI
111624102217	TAMIL MANI B
111624102222	THARUN

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **SURENDAR HARIPRASAD, E THAMILNIDHI, TAMIL MANI B and THARUN**, bearing **Register Numbers 111624102215, 111624102219, 111624102217 and 111624102222** respectively, of **3rd Year, 5th Semester, Department of Computer Science and Engineering**, have successfully completed the mini project entitled **"Decidable Language Analyzer"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student

SURENDAR HARIPRASAD

E THAMILNIDHI

TAMIL MANI B

THARUN

Project Guide

ABSTRACT

The Theory of Computation separates the problems that an algorithm can solve completely from those that no algorithm can ever solve. A language is decidable when some Turing machine halts on every input and correctly reports whether the input belongs to it. Many questions about regular and context-free languages are decidable, whereas questions such as the halting problem are not. These ideas are normally taught on paper, which makes them difficult to experience. This mini project presents the Decidable Language Analyzer, a software tool in which a user describes a language as a regular expression, a DFA, an NFA, a context-free grammar or a Turing machine, chooses a question about it, and receives both an answer and an explanation of whether that question is decidable. The analyzer implements the classical deciders from first principles. For finite automata it provides the Thompson and subset constructions, simulation, reachability-based emptiness testing, product-construction equivalence testing, finiteness testing and Moore minimisation. For context-free grammars it provides Chomsky normal form conversion, the CYK parsing algorithm, marking-based emptiness testing and cycle-based finiteness testing. For Turing machines it provides a simulator that separates acceptance, rejection, provable looping through a repeated configuration and step-limit timeouts. For undecidable questions such as grammar equivalence, grammar universality, Turing machine emptiness and halting, it runs a bounded search and labels the outcome as conclusive or inconclusive. A built-in catalogue of 23 classical languages records the decidability class and the proof idea for each. The tool is written in Python using only the standard library, offers a command-line interface and a browser interface, and is verified by 19 automated unit tests. It shows concretely that decidable problems yield definite answers, while undecidable ones can yield only partial evidence.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	5
2	Objectives	7
3	Tools and Technologies	8
4	Architecture Diagram	9
5	Methodology	11
6	Results	15
7	Conclusion	23
8	Reference	24

1. INTRODUCTION

1.1 Background

The Theory of Computation asks a deceptively simple question: what can a computer do, and what can it never do? To answer it, the field defines abstract machines of increasing power and studies the languages they accept. Finite automata and regular expressions describe the regular languages; pushdown automata and context-free grammars describe the context-free languages; and Turing machines describe everything that can be computed by an algorithm. This ladder of power is known as the Chomsky hierarchy.

A language is Turing-recognizable if some Turing machine accepts exactly its members, but that machine is allowed to run forever on non-members. A language is decidable if some Turing machine, called a decider, halts on every input and so always gives a definite yes or no answer. Every decidable language is recognizable, but not the other way round: the classical counter-example is the halting problem, proved undecidable by Alan Turing in 1936. A language is decidable exactly when both it and its complement are recognizable.

Decidability is also a property of questions about machines, not only of languages. Asking whether a given DFA accepts a given string is decidable, asking whether two DFAs are equivalent is decidable, but asking whether two context-free grammars are equivalent, or whether a Turing machine halts, is not. Table 1.1 summarises how the answer changes as the computational model becomes more powerful.

Table 1.1 Decidability of common questions for each model

Question	Regular (DFA / NFA / regex)	Context-free (CFG / PDA)	Turing machine
Membership ($w \in L$?)	Decidable	Decidable	Undecidable (recognizable)
Emptiness ($L = \varnothing$?)	Decidable	Decidable	Undecidable (co-recognizable)
Finiteness	Decidable	Decidable	Undecidable
Universality ($L = \Sigma^*$?)	Decidable	Undecidable	Undecidable
Equivalence ($L_1 = L_2$?)	Decidable	Undecidable	Undecidable

1.2 Problem Statement

Textbooks present these results as theorems and proofs. Students rarely see the corresponding algorithms run, and almost never see an undecidable question being attempted and failing. There is no simple tool that takes an arbitrary language description, decides whether a chosen question about it is decidable, and, when it is, executes the decision procedure and shows its

working. The problem addressed by this project is to build such a tool: the Decidable Language Analyzer.

1.3 Motivation

- Decidability is abstract; seeing a decider halt with a witness, and a recogniser loop, makes it concrete.
- The same question (for example, equivalence) is easy for DFAs and impossible for grammars; a tool that shows both side by side explains why the model matters.
- Hand-tracing CYK tables, subset constructions and Turing machine configurations is slow and error-prone; automation lets learners check their own work.
- Undecidability is rarely demonstrated honestly. The analyzer reports bounded searches as inconclusive rather than pretending to solve unsolvable problems.

1.4 Scope

The project covers deterministic and nondeterministic finite automata (with ε-moves), regular expressions, context-free grammars, and single-tape deterministic Turing machines. It supports the questions membership, emptiness, universality, finiteness, equivalence and minimisation where they apply, and reports the decidability class of every question it is asked. Pushdown automata, multi-tape and nondeterministic Turing machines, and graphical state diagrams are outside the scope of this mini project.

2. OBJECTIVES

The objectives of the project are:

- To study the hierarchy regular ⊂ context-free ⊂ decidable ⊂ recognizable and the meaning of decidability.
- To implement the standard conversion algorithms: regular expression → NFA → DFA, and CFG → Chomsky normal form.
- To implement the classical deciders: A_DFA, E_DFA, ALL_DFA, EQ_DFA, FIN_DFA, A_CFG (CYK), E_CFG and FIN_CFG.
- To build a Turing machine simulator that distinguishes accepting, rejecting, provably looping and undetermined runs.
- To handle undecidable questions (HALT_TM, E_TM, ALL_CFG, EQ_CFG) with clearly labelled bounded searches instead of false answers.
- To maintain a catalogue of 23 classical languages with their decidability class and proof idea.
- To provide a command-line interface and a browser interface so that results can be explored interactively.
- To verify correctness with an automated test suite (19 tests) and a set of demonstration cases.

3. TOOLS AND TECHNOLOGIES

The analyzer is deliberately dependency-free: every algorithm is written from first principles in Python so that the code mirrors the textbook constructions. Table 3.1 lists what was used.

Table 3.1 Tools and technologies used

Category	Technology	Purpose
Language	Python 3	All automata, grammar and Turing machine algorithms
Libraries	Python standard library only (re, collections, itertools, argparse, json, http.server, unittest)	Parsing, data structures, command-line interface, web server, testing
User interface	HTML5, CSS3 and vanilla JavaScript	Single-page browser interface calling a JSON API
Testing	unittest	19 automated tests across the four modules and the engine
Report assets	Matplotlib, headless Google Chrome	Architecture diagrams and interface screenshots
Platform	macOS / any OS with Python 3	Development and execution

3.1 Running the project

```
# browser interface at http://127.0.0.1:8000
python3 main.py serve --port 8000

# run every built-in example
python3 main.py demo

# list the classical (un)decidable languages
python3 main.py catalogue

# analyse one language and question
python3 main.py analyze --model cfg --problem membership \
        --spec "S -> a S b | ε" --input aabb

# run the test suite
python3 -m unittest discover -s tests
```

4. ARCHITECTURE DIAGRAM

The analyzer is organised in four layers (Figure 4.1). The presentation layer is either the browser interface or the command line. The service layer exposes a small JSON API. The analyzer engine dispatches each request to the module for the chosen computational model, attaches the decidability class from the catalogue, and formats the result. The model modules hold the actual algorithms. Modules coloured green contain only deciders; the Turing machine module, coloured red, contains recognisers and bounded searches.

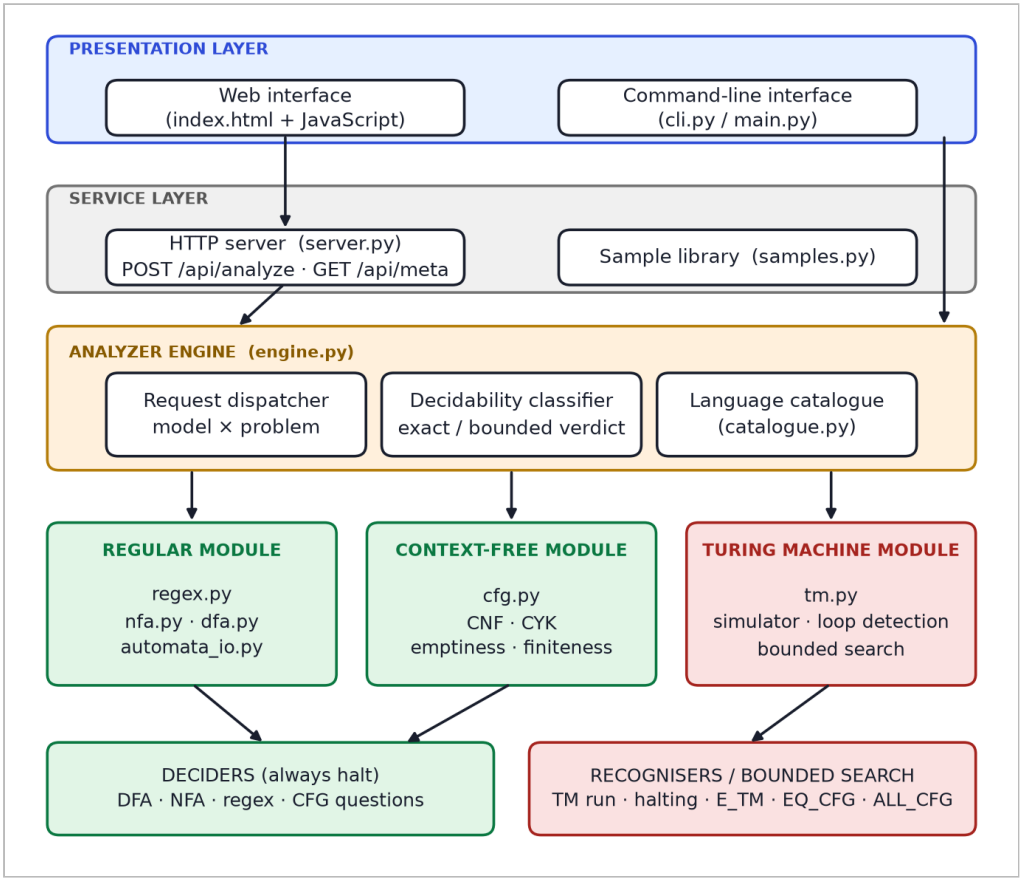


Figure 4.1 Layered architecture of the Decidable Language Analyzer

Table 4.1 Module responsibilities

File	Responsibility
regex.py	Regular-expression parser and Thompson construction (regex → ε-NFA)
nfa.py	NFA with ε-moves, closure, simulation and subset construction (NFA → DFA)
dfa.py	DFA simulation, emptiness, universality, finiteness, equivalence, minimisation
automata_io.py	Text format reader for DFAs and NFAs with validation
cfg.py	Grammar parser, ε/unit/useless-symbol removal, Chomsky normal form, CYK, emptiness, finiteness
tm.py	Turing machine parser, simulator with loop detection, decider test
catalogue.py	Knowledge base of 23 classical languages with class and proof idea

File	Responsibility
engine.py	Dispatcher: (model, question) → procedure → structured result
server.py / cli.py	HTTP JSON API and command-line front ends
web/index.html	Browser interface

4.1 Decision flow

Figure 4.2 shows how a single request is handled. After parsing, the engine looks the (model, question) pair up in the catalogue. If the question is decidable, the decider is run and the answer is exact. If it is undecidable, the engine runs a bounded search or simulation; a witness found by the search gives a conclusive answer, while an empty search gives an explicitly inconclusive one.

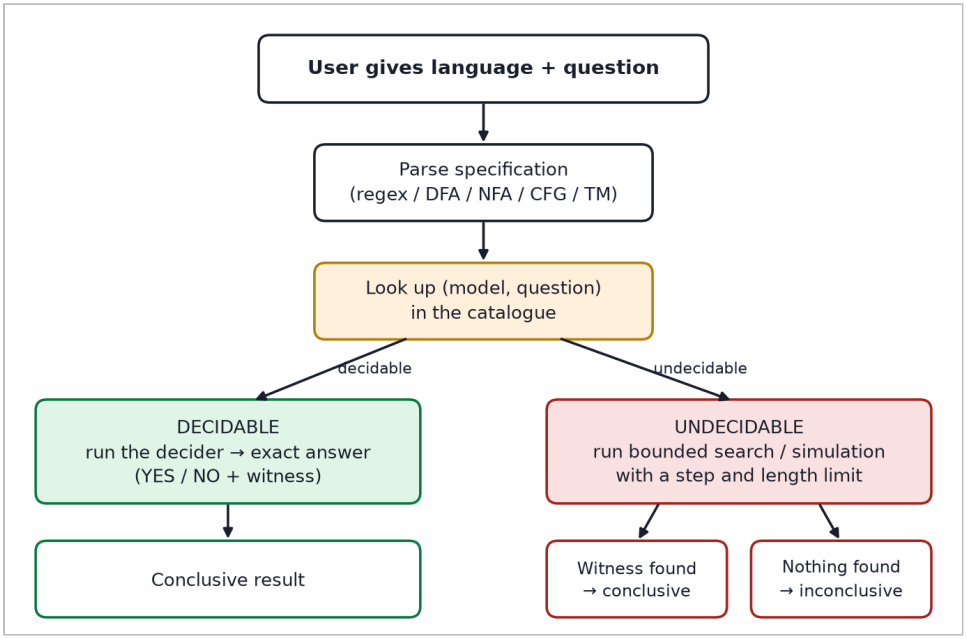


Figure 4.2 Request handling and the exact / bounded distinction

5. METHODOLOGY

5.1 Overall approach

Development followed the structure of the theory itself. First the model-independent pieces were built (parsers and result structures), then one module per computational model in increasing order of power, then the engine that connects them, and finally the interfaces. Each algorithm was written to follow its textbook definition, and each was tested against hand-verified examples before being integrated.

5.2 Input formats

Every model has a plain-text format. Regular expressions use |, *, +, ?, parentheses, ε and backslash escapes. Automata list a start state, accepting states and one transition per line. Grammars are written as productions. Turing machines declare their states and give transitions of the form q a → r b D.

```
# DFA
start: q0
accept: q0
q0 0 q1
q0 1 q0
q1 0 q0
q1 1 q1
```

```
# Context-free grammar
S -> a S b | ε
```

```
# Turing machine (excerpt)
states: q0 q1 q2 q3 qa qr
input: 0 1
start: q0
accept: qa
reject: qr
q0 0 -> q1 X R
```

5.3 Regular languages

Regular expressions are parsed by recursive descent into a syntax tree and converted to an ε-NFA by Thompson's construction, which adds at most two states per operator. NFAs are converted to DFAs by the subset construction: each DFA state is the ε-closure of a set of NFA states. All regular questions are then answered on the DFA, so a single set of deciders serves regex, NFA and DFA input.

Each decider is a terminating graph algorithm:

- A_DFA – run the DFA on w; it halts after |w| steps.
- E_DFA – breadth-first search from the start state; L = ∅ exactly when no accepting state is reachable. The search also yields the shortest member.
- ALL_DFA – complement the DFA by swapping accepting and non-accepting states of the completed automaton, then apply E_DFA.
- EQ_DFA – search the product automaton for a reachable pair of states where exactly one component accepts; the path to it is a shortest distinguishing string.
- FIN_DFA – L is infinite exactly when a state that is both reachable and able to reach an accepting state lies on a cycle; this is tested by topological elimination.
- Minimisation – Moore's partition refinement after removing unreachable states, which yields the Myhill–Nerode equivalence classes.

```
EQ_DFA(A, B):
    complete both automata over the union alphabet
    queue ← [(startA, startB)]
    while queue not empty:
        (p, q) ← dequeue                        # pairs are visited in order of
string length
        if (p ∈ FA) ≠ (q ∈ FB): return NOT-EQUIVALENT, path(p, q)
        for each symbol c: enqueue (δA(p,c), δB(q,c)) if unseen
    return EQUIVALENT
```

5.4 Context-free languages

A grammar is first simplified and converted to Chomsky normal form (CNF), in which every rule is A → BC or A → a. The conversion removes ε-rules (by adding every variant of a rule with nullable symbols deleted), unit rules A → B (by copying the rules of B), non-generating and unreachable variables, and finally replaces terminals inside long rules by new variables and splits long rules into binary ones. Whether the start variable is nullable is remembered separately so that the empty string is handled correctly.

Membership is decided with the Cocke–Younger–Kasami (CYK) algorithm. It fills a triangular table in which cell (i, l) holds the variables that derive the substring of length l starting at position i. Back-pointers allow a parse tree to be read off. Because every derivation of a string of length n in a CNF grammar has exactly 2n − 1 steps, the algorithm always terminates, which is the argument that A_CFG is decidable.

```
CYK(G in CNF, w = w1 … wn):
    for i in 1..n: T[i][1] ← { A | A → wi ∈ G }
    for l in 2..n:
        for i in 1..n-l+1:
            for k in 1..l-1:
                for each rule A → B C:
                    if B ∈ T[i][k] and C ∈ T[i+k][l-k]: add A to T[i][l]
    return S ∈ T[1][n]
```

Emptiness (E_CFG) is decided by repeatedly marking variables that have a rule whose symbols are all terminals or already marked; L(G) = ∅ exactly when the start variable is never marked. Finiteness (FIN_CFG) is decided on the useful part of the CNF grammar: L(G) is infinite exactly when the graph with an edge A → B for every rule A → BC or A → CB contains a cycle.

5.5 Turing machines

The simulator executes a deterministic single-tape Turing machine whose tape is unbounded to the right. A missing transition means an implicit move to the reject state. Because the halting problem is undecidable, a run has four possible outcomes rather than two:

Table 5.1 Outcomes of a simulated run

Outcome	Meaning	Conclusive?
ACCEPT	The machine reached its accept state	Yes
REJECT	The machine reached its reject state or had no applicable transition	Yes
LOOP	A configuration (state, head position, tape contents) repeated, so the machine can never halt	Yes
TIMEOUT	The step budget was used up; the machine may loop or may simply be slow	No

Loop detection stores every configuration seen; a repeat proves non-termination because the machine is deterministic. This catches cycling machines but not machines that run forever by moving ever further into blank tape, which is why TIMEOUT cannot be turned into a verdict. A decider test runs a machine on every string up to a chosen length and reports any input on which it fails to halt; passing the test is evidence of a decider but never a proof, since deciding whether a machine is a decider is itself undecidable.

5.6 Undecidable questions

Four questions offered by the analyzer have no decision procedure: HALT_TM, E_TM, ALL_CFG and EQ_CFG. For each, the engine exploits the fact that one direction is recognizable. A non-empty Turing machine language has a witness that enumeration will eventually reach; a grammar that is not universal, or two grammars that are not equivalent,

have a shortest counter-example that enumeration will eventually reach. The analyzer therefore searches all strings up to a user-chosen length. A counter-example is reported as a conclusive answer; an empty search is reported as INCONCLUSIVE and marked as a bounded search, never as proof.

5.7 Decidability catalogue

A knowledge base of 23 languages (13 decidable) records the formal definition, the class (decidable; undecidable but recognizable; undecidable but co-recognizable; or neither) and a one-line proof idea for each. It includes A_DFA, A_NFA, A_REX, E_DFA, EQ_DFA, A_CFG, E_CFG, A_TM, HALT_TM, E_TM, EQ_TM, ALL_CFG, EQ_CFG, the Post Correspondence Problem and the diagonal language. Every analysis result is annotated from this catalogue.

5.8 Complexity of the procedures

Table 5.2 Time complexity of the implemented deciders

Problem	Procedure	Time
A_DFA	Simulation	$O(\|w\|)$
E_DFA, ALL_DFA, FIN_DFA	Graph search / cycle detection	$O(\|Q\|\cdot\|\Sigma\|)$
EQ_DFA	Product automaton search	$O(\|Q_1\|\cdot\|Q_2\|\cdot\|\Sigma\|)$
Minimisation	Moore partition refinement	$O(\|Q\|^2\cdot\|\Sigma\|)$
NFA → DFA	Subset construction	O(2^|Q|) worst case
A_CFG	CNF + CYK	$O(\|w\|^3\cdot\|G\|)$
E_CFG	Fixed-point marking	$O(\|G\|\cdot\|V\|)$
FIN_CFG	Cycle detection in CNF graph	$O(\|G\|^2)$
Bounded searches	Enumerate strings up to length n	$O(\|\Sigma\|^n)$ strings

5.9 Testing methodology

The test suite contains 19 unit tests. They check regular-expression operators, membership, emptiness, equivalence with counter-examples, finiteness, minimisation (the minimal DFA must be equivalent to the original), subset construction against direct NFA simulation, CNF/CYK on four grammars, grammar emptiness and finiteness, exact language enumeration, three Turing machines (including loop detection and timeout), and error handling for malformed input. A final test runs every one of the 19 built-in samples through the engine.

6. RESULTS

6.1 Test results

All 19 unit tests pass. The 19 built-in examples run without error through the engine, the command-line interface and the browser interface. Table 6.1 lists 24 demonstration cases and the exact output of the analyzer for each.

Table 6.1 Demonstration cases and analyzer output

#	Model	Question	Language	Input	Analyzer answer	Result
1	REGEX	Membership	(0|1)*01	1101	ACCEPTED: '1101' ∈ L	Exact
2	REGEX	Membership	(0|1)*01	110	REJECTED: '110' ∉ L	Exact
3	DFA	Membership	even number of 0s	1001	ACCEPTED: '1001' ∈ L	Exact
4	NFA	Membership	3rd symbol from end is 1	0100	ACCEPTED: '0100' ∈ L	Exact
5	REGEX	Minimization	(a|b)*ab(a|b)*	—	MINIMAL DFA has 3 states	Exact
6	REGEX	Equivalence	(a|b)* vs (a*b*)*	—	EQUIVALENT: L(A) = L(B)	Exact
7	REGEX	Equivalence	a* vs a*b?	—	NOT EQUIVALENT: 'b' is accepted only by B (second)	Exact
8	REGEX	Finiteness	a|ab|abb	—	FINITE: |L| = 3 → a, ab, abb	Exact
9	CFG	Membership	S → aSb | ε	aaabbb	ACCEPTED: 'aaabbb' ∈ L(G)	Exact
10	CFG	Membership	S → aSb | ε	aaabb	REJECTED: 'aaabb' ∉ L(G)	Exact
11	CFG	Membership	palindromes over {a,b}	abbaabba	ACCEPTED: 'abbaabba' ∈ L(G)	Exact
12	CFG	Membership	arithmetic expressions	a+a*(a+a)	ACCEPTED: 'a+a*(a+a)' ∈ L(G)	Exact
13	CFG	Emptiness	S → AB, A → aA, B → b	—	EMPTY: L(G) = ∅	Exact
14	CFG	Finiteness	S → aA | b, A → cc | d	—	FINITE	Exact
15	CFG	Universality	S → aS | bS | ε	—	INCONCLUSIVE: L(G) contains all strings up to length 6, but ALL_CFG is undecidable	Bounded
16	CFG	Universality	S → aSb | ε	—	NOT UNIVERSAL: 'a' ∉ L(G)	Exact
17	CFG	Equivalence	a^nb^n vs a^nb^{2n}	—	NOT EQUIVALENT: 'ab' is generated only by G (first)	Exact
18	TM	Run (A_TM)	decider for 0^n1^n	000111	ACCEPTED: M halts in the accept state	Exact
19	TM	Run (A_TM)	decider for 0^n1^n	00111	REJECTED: M halts in the reject state	Exact
20	TM	Run (A_TM)	decider for $a^nb^nc^n$	aabbcc	ACCEPTED: M halts in the accept state	Exact

#	Model	Question	Language	Input	Analyzer answer	Result
21	TM	Run (A_TM)	decider for $a^n b^n c^n$	aabbc	REJECTED: M halts in the reject state	Exact
22	TM	Halting	recognizer, loops on 0	0	DOES NOT HALT (repeated configuration)	Exact
23	TM	Decider check	decider for $0^n 1^n$	—	HALTS ON ALL TESTED INPUTS (length ≤ 6) - evidence of a decider, not a proof	Bounded
24	TM	Decider check	recognizer, loops on 0	—	NOT A DECIDER: fails to halt on '0' (LOOP)	Exact

Cases 1 to 14 show the decidable questions returning definite answers, with a shortest counter-example whenever the answer is negative. Cases 15 to 17 concern undecidable grammar questions. In case 15 the grammar S → aS | bS | ε really is universal, but the analyzer can only report that nothing was found up to the bound; in cases 16 and 17 a counter-example is found, so the answer is conclusive. Cases 18 to 22 run Turing machines, and cases 23 and 24 show the decider test separating a machine that always halts from one that does not.

6.2 Worked example: CYK on aabb

For the grammar S → aSb | ε the analyzer converts to the following Chomsky normal form and fills the CYK table below. The start variable S appears in the top cell, so aabb is accepted.

```
S -> T1 T2 | T1 Y1
T1 -> a
T2 -> b
Y1 -> S T2
(plus S -> ε, allowed only for the start variable)
```

Table 6.2 CYK table for w = aabb (row l = substrings of length l)

	1:a	2:a	3:b	4:b
len 4	{S}			
len 3	{}	{Y1}		
len 2	{}	{S}	{}	
len 1	{T1}	{T1}	{T2}	{T2}

```
S
├─ T1
│  └─ a
└─ Y1
   ├─ S
   │  ├─ T1
   │  │  └─ a
   │  └─ T2
   │     └─ b
   └─ T2
      └─ b
```

6.3 Worked example: DFA minimisation

The regular expression (a|b)*ab(a|b)* produces a DFA with 7 states by Thompson and subset construction. Partition refinement merges them into the 3-state minimal DFA below: q0 (no progress), q1 (last symbol was a) and q2 (ab has been seen, accepting).

Table 6.3 Minimal DFA for strings containing ab

State	a	b
→ q0	q1	q0
q1	q1	q2
* q2	q2	q2

6.4 Worked example: Turing machine run

The decider for 0^n1^n marks one 0 as X and one 1 as Y per pass. The configuration trace for the input 0011 is shown below (the state is written in square brackets in front of the scanned symbol).

```
[q0]0011
X[q1]011
X0[q1]11
X[q2]0Y1
[q2]X0Y1
X[q0]0Y1
XX[q1]Y1
XXY[q1]1
XX[q2]YY
X[q2]XYY
XX[q0]YY
XXY[q3]Y
XXYY[q3]_
XXYY_[qa]_
```

In contrast, the recogniser from the third sample loops on the input 0. The analyzer proves this because a configuration repeats: it reports “DOES NOT HALT (repeated configuration)”.

6.5 Emptiness of a grammar

For the grammar S → AB, A → aA, B → b the marking procedure runs as follows.

```
Round 1: marked B
Start variable S was never marked.
→ EMPTY: L(G) = ∅
```

A can never terminate (A → aA only), so it is never marked and S is never marked either.

6.6 Browser interface

Figures 6.1 to 6.4 show the browser interface. The badge at the top of the result panel states the decidability class of the question; a second badge appears whenever the result comes from a bounded search and is therefore not a proof.

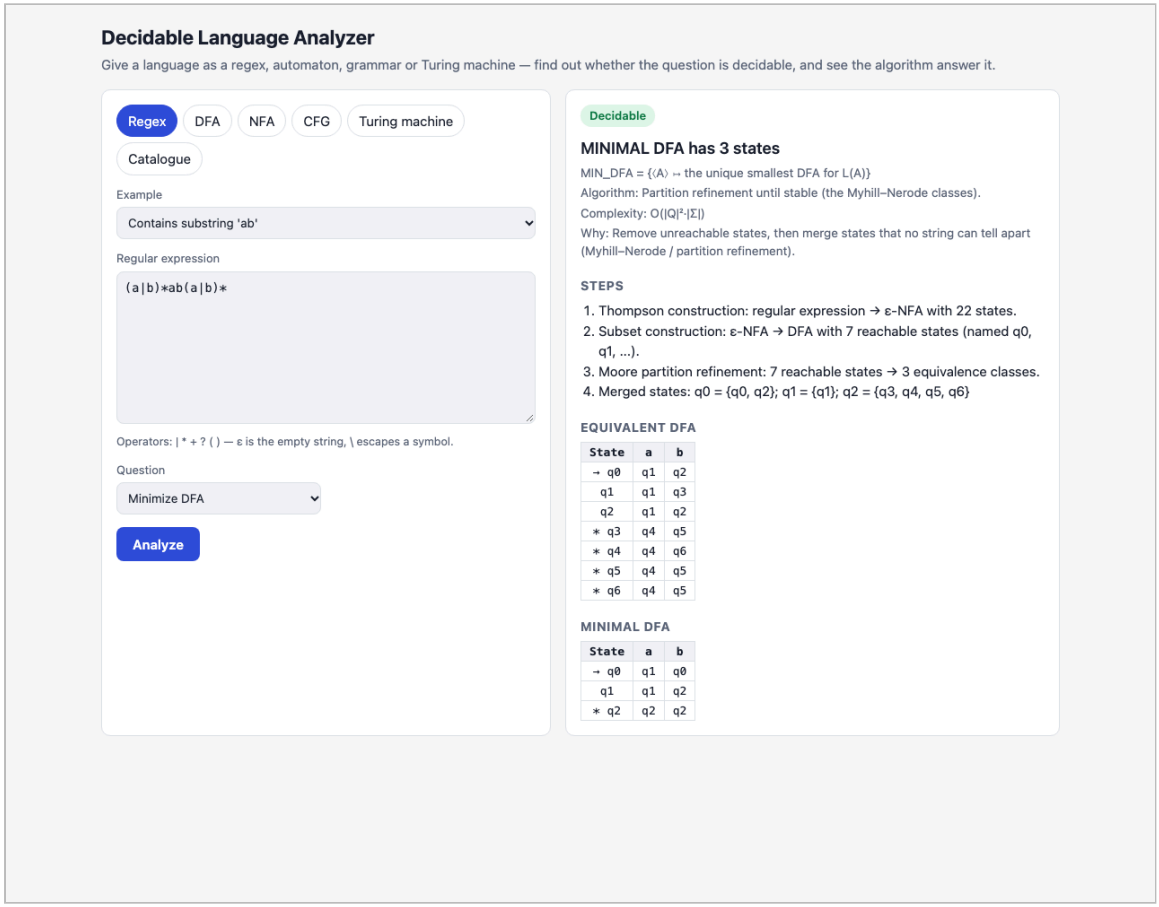


Figure 6.1 Regular expression: minimisation of (a|b)*ab(a|b)*

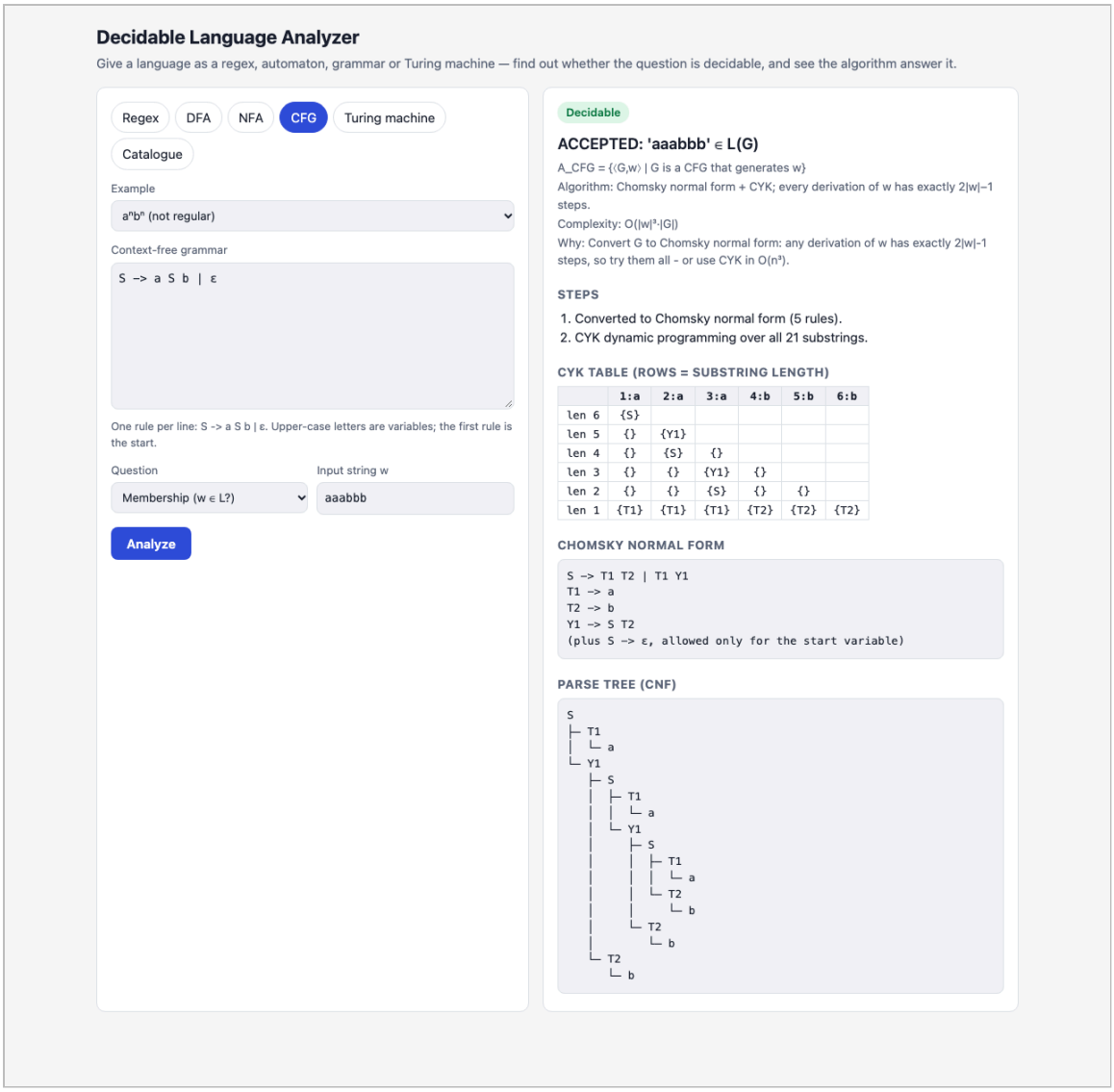


Figure 6.2 Context-free grammar: CYK membership test with CNF and CYK table

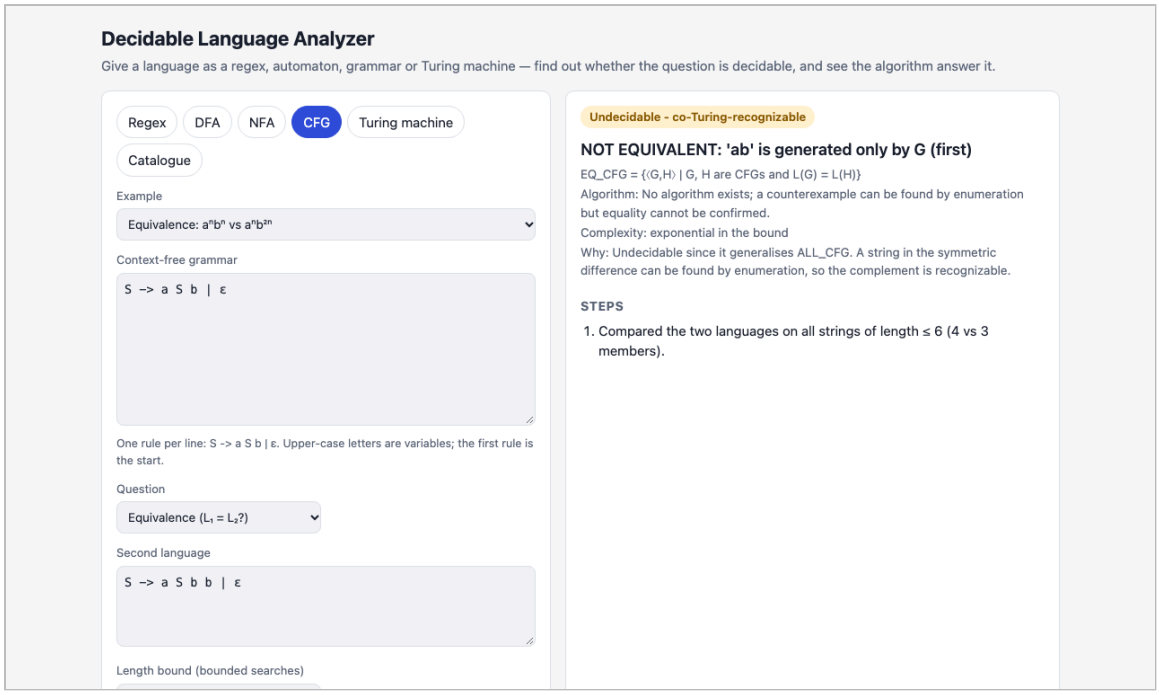


Figure 6.3 Grammar equivalence: undecidable question, counter-example found

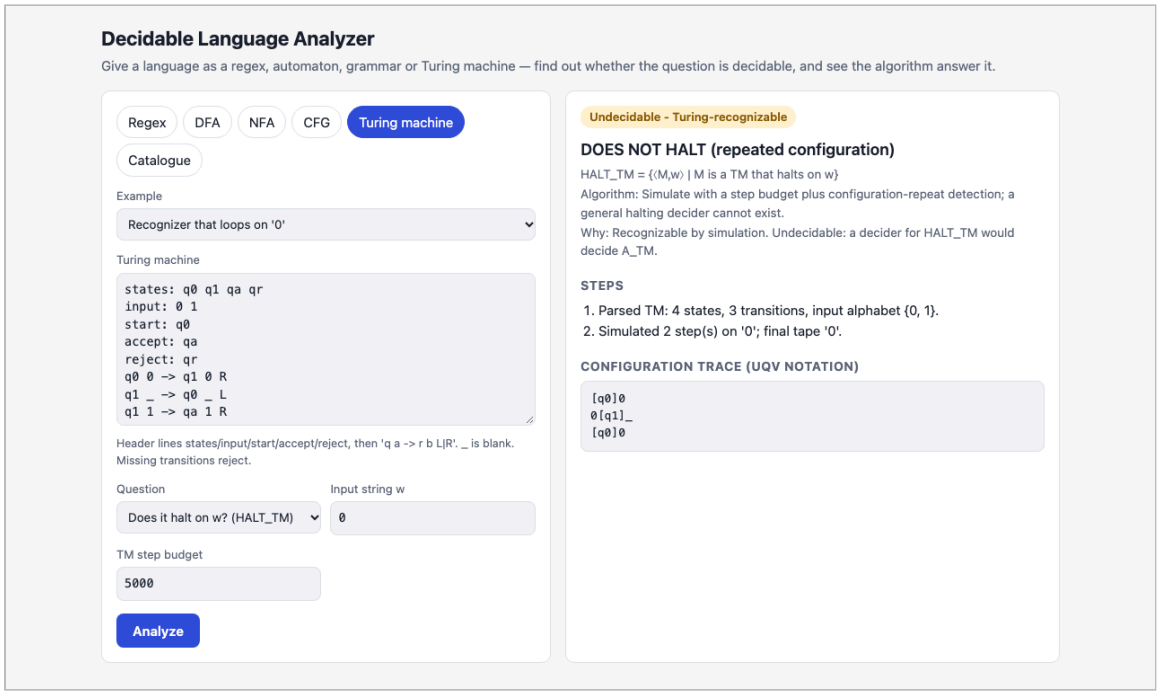


Figure 6.4 Turing machine: halting check proves that the machine loops

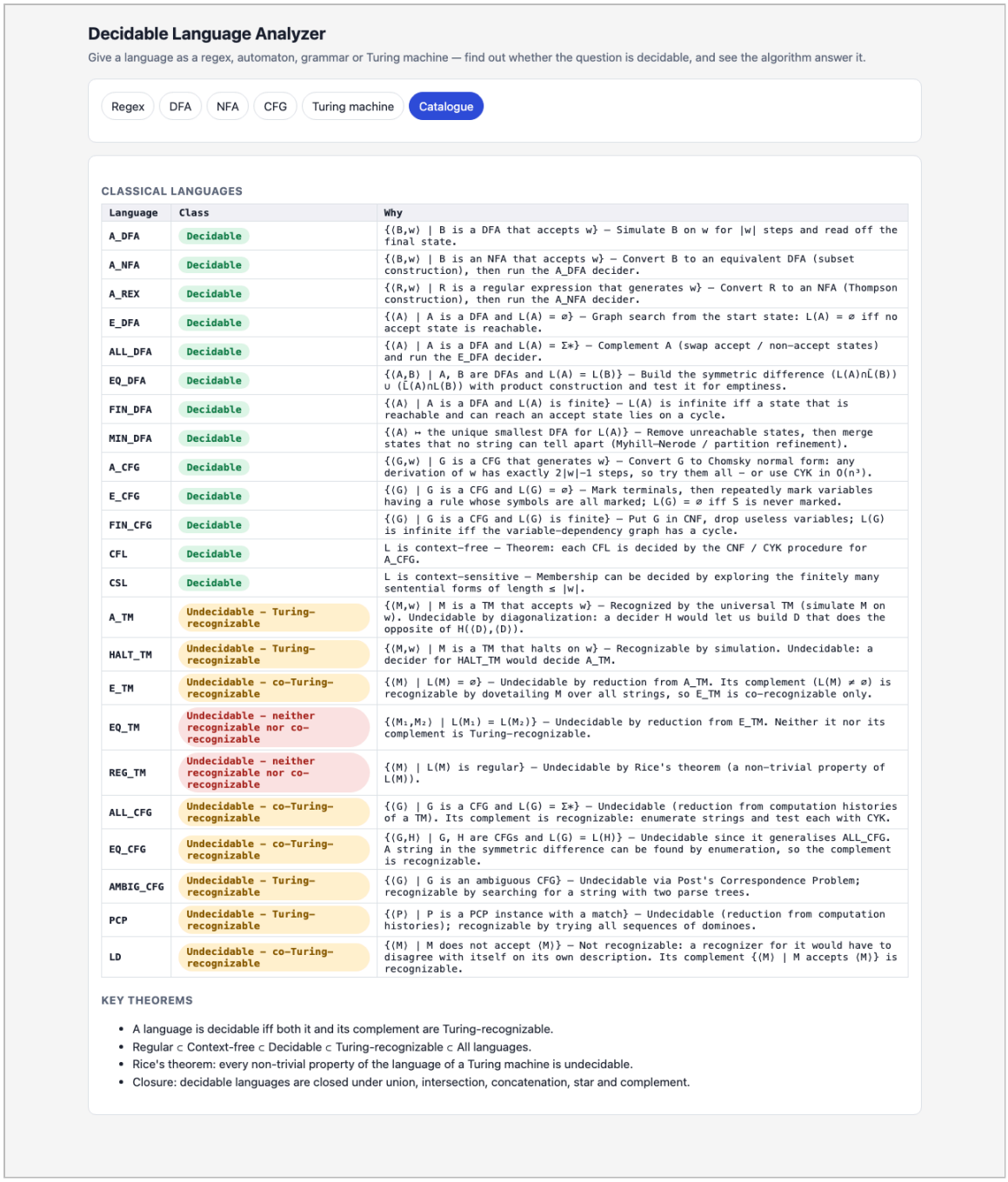


Figure 6.5 Catalogue of classical decidable and undecidable languages

6.7 Performance

Table 6.4 shows the CYK decider on the arithmetic-expression grammar for strings a+a+…+a of growing length, and Table 6.5 shows the cost of the subset construction for the regular expressions (a|b)*a(a|b)^k, whose minimal DFA needs exactly 2^(k+1) states. Times were measured on the development machine and are indicative only.

Table 6.4 CYK running time

String length	Result	Time (ms)
11	ACCEPTED	0.1
21	ACCEPTED	0.3
41	ACCEPTED	1.8
81	ACCEPTED	7.0

Table 6.5 State growth of the subset construction

k	NFA states	DFA states	Minimal DFA states	Time (ms)
2	24	9	8	0.2
4	36	33	32	0.9
6	48	129	128	3.6
8	60	513	512	17.7
10	72	2049	2048	76.4

6.8 Discussion

- Decidable questions always returned an exact answer, with a witness or counter-example for negative cases.
- The running time of CYK grows polynomially with the string length, consistent with the $O(n^3)$ bound.
- The minimal DFA size doubles with each increment of k, confirming that the exponential blow-up of the subset construction cannot be avoided in general.
- For the four undecidable questions the analyzer never claims more than it can prove: a negative search is always labelled inconclusive.
- The sample machine for $a^n b^n c^n$ is accepted correctly even though this language is not context-free, illustrating that decidable languages extend beyond the context-free ones.

7. CONCLUSION

The Decidable Language Analyzer turns the central results of decidability theory into something that can be run. For regular and context-free languages it decides membership, emptiness, finiteness and (for regular languages) universality, equivalence and minimisation with exact answers and witnesses. For Turing machines it provides a faithful simulator that proves non-termination whenever a configuration repeats. For questions that provably have no algorithm it falls back to bounded search and reports the limits of its own answer. The tool therefore demonstrates, on the same interface, why decidable problems yield definite answers while undecidable ones yield only partial evidence.

7.1 Limitations

- Only deterministic single-tape Turing machines are supported, and loop detection finds repeated configurations but not machines that drift forever into blank tape.
- Bounded searches are exponential in the length bound, so the bound is capped at small values.
- Grammar variables are single upper-case letters with optional digits or primes; regular expressions have no character classes.
- The subset construction is exponential in the worst case (Table 6.5).

7.2 Future scope

- Add pushdown automata and nondeterministic or multi-tape Turing machines.
- Draw automata as state diagrams and animate Turing machine tapes.
- Visualise reductions such as A_TM ≤ HALT_TM to teach undecidability proofs.
- Add a Rice's-theorem helper that classifies properties of recognised languages.
- Export results as printable worksheets for classroom use.

8. REFERENCE

- M. Sipser, Introduction to the Theory of Computation, 3rd ed., Cengage Learning, 2013.
- J. E. Hopcroft, R. Motwani and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed., Pearson, 2006.
- A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem," Proceedings of the London Mathematical Society, series 2, vol. 42, pp. 230–265, 1936.
- K. Thompson, "Programming techniques: Regular expression search algorithm," Communications of the ACM, vol. 11, no. 6, pp. 419–422, 1968.
- D. H. Younger, "Recognition and parsing of context-free languages in time n^3," Information and Control, vol. 10, no. 2, pp. 189–208, 1967.
- E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies, Princeton University Press, 1956, pp. 129–153.
- H. G. Rice, "Classes of recursively enumerable sets and their decision problems," Transactions of the American Mathematical Society, vol. 74, pp. 358–366, 1953.
- Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/.